-- CONHECIMENTOS ESPECÍFICOS --

```
estrutura S
S.topo=0

estrutura-vazia(S){
     se (S.topo=0)
          retorna Verdadeiro;
     fim se
}
EMPURRA (S,x){
     S.topo = S.topo+1;
     S[S.topo] = x;
}
PUXA (S)
     se (Stack-Empty(S))
          escreva_erro("Underflow");
     senao
          S.topo = S.topo-1;
          retorna S[S.topo+1];
     fim se
```

Considerando a estrutura `S` precedente, inicialmente vazia e armazenada no arranjo `S[1 … 6]`, julgue o próximo item, a respeito de construção de algoritmos e estrutura de dados.

51 O resultado final das operações na sequência `EMPURRA(S, 4)`, `EMPURRA (S, 1)`, `EMPURRA (S, 3)`, `PUXA(S)`, `EMPURRA (S, 8)` e `PUXA (S)` é `S[4,1,3,8]`.

Acerca de *hashing*, julgue o item subsecutivo.

52 Conforme a premissa do *hashing* uniforme simples, cada chave tem igual probabilidade de passar para qualquer das *m* posições por uma operação de *hash*, independentemente da posição que qualquer outra chave tenha ocupado após o *hash*. No método de divisão para criar funções *hash*, é mapeada uma chave *k* para uma de *m* posições, tomando-se o resto da divisão de *k* por *m*, de modo que a função *hash* é $h(k) = k \bmod m$.

No que concerne a conceitos de algoritmos e blocos de comandos, julgue os itens seguintes.

53 As funções são um bloco de código ou comandos constituindo um conjunto específico de instruções repetíveis, que recebem uma ou mais entradas e produzem alguma saída.

54 Algoritmo é uma lista de instruções que conduzem ações especificadas, passo a passo, em rotinas embasadas em *hardware* ou *software*.

Com relação a estruturas de controle, seleção, repetição e desvio, julgue os itens a seguir.

55 Um *loop* que sempre se repetirá em um determinado número de vezes é representado pelo código `SE ENTÃO SENÃO FIM SE`.

56 Ao final da execução do algoritmo a seguir, o valor `0` será apresentado.

```
constante A = 50
enquanto (A > 0)
     A -= 5;
fim enquanto
escreva (A);
```

57 No algoritmo a seguir, o comando `x = x + 10` é executado quatro vezes.

```
x = 10;
para (y = 40; y < 100; y = y + 16)
     x = x + 10;
fim para
escreva(x);
```

58 Se os valores `a = 3`, `b = 4` e `c = 8` forem entradas do algoritmo a seguir e o sistema no qual o algoritmo for executado utilizar números decimais com quatro casas de precisão, então a execução do referido algoritmo apresentará em tela o resultado `10.3923`.

```
sp = (a + b + c)/2;
ar = sp*(sp - a)*(sp - b)*(sp - c);



se (ar < 0)
     escreva ("Não é possível obter
resultado.");
senão
     escreva ("Resultado: ");
     escreva(raiz_quadrada(ar));
fimse
```

59 Após executado, o algoritmo a seguir apresentará `720` como resultado final.

```
função f(x)
  se (x == 0 ou x == 1)
     retorna 1;
  fimse
  retorna f(x - 1)*x
fimfunção

função func(a)
  se (a == 0)
     retorna 2
  fimse
  retorna a + f(a - 1)
fimfunção

escreva(func(6));
```

A respeito de recursividade, julgue o próximo item.

60 Uma grande vantagem da utilização da recursividade é o baixo consumo de memória.

```
real c, b, d;
real x, y;
c = 5;
b = 8;
d = 3;
x = (c < b) ou (b < d) e (c < d);
y = ((c < b) ou (b < d)) e (c < d);
escreva(x);
escreva(y);
```

Com base no algoritmo precedente, julgue o item a seguir, relativo a operadores e expressões.

61 Após a execução desse algoritmo, os valores das variáveis `x` e `y` serão diferentes.

Julgue os itens que se seguem, relativos a conceitos de avaliação de expressões.

62 Após o algoritmo a seguir ser executado, os valores das variáveis `c` e `d` serão iguais.

```
real d
real c
d = 6 / -2 + -3 * -2 - 3
escreva (d)
c = 6 / -2 + 3 * 2 - 3
escreva(c)
```

63 Após o algoritmo a seguir ser executado, o valor da variável `soma1` será maior que o da variável `soma2`.

```
vetor a[7];
real soma1, soma2;
inteiro i;
a = [1,3,9,27,81,243,729];}
soma1 = 0;
i = 0;
enquanto (i < 7) faça
     soma1 = soma1 + a[i]
     i = i + 1
fim enquanto
soma2 = 1 * (1-3^7)/(1-3)

escreva(soma1)
escreva(soma2)
```

Julgue os próximos itens, relacionados a métodos de ordenação e pesquisa.

64 O *selection sort* (ordenamento por seleção) faz sempre (n^2–n)/2 comparações, independentemente do vetor inicial.

65 Para que a pesquisa binária possa ser utilizada para localizar uma entrada, é requisito que os dados estejam em vetor cujo tamanho seja múltiplo de dois.

A respeito de programação estruturada e orientada a objetos, julgue os itens subsequentes.

66 Em programação orientada a objetos, abstração é a propriedade que permite que um método de determinado nome tenha comportamentos distintos, em função de diferentes parâmetros recebidos.

67 A programação estruturada é formada por código em um único bloco para a criação de estruturas simples nos programas, e as sub-rotinas e funções são formas de facilitar a manutenção de programas.

A respeito de arquitetura de *software*, julgue os itens a seguir.

68 As técnicas de componentização de *software* são usadas para dividir um sistema de *software* em componentes independentes e reutilizáveis, o que ajuda a diminuir a complexidade do sistema.

69 As arquiteturas voltadas para o desenvolvimento de microsserviços apresentam uma lógica simétrica, em que o uso de contêineres é considerado chave para tornar um processo mais eficiente.

70 A arquitetura orientada a eventos aborda diferentes tópicos, como modelagem de eventos, fluxo de eventos, gerenciamento de estado, integração de sistemas e operações, e é frequentemente utilizada em sistemas centralizados e em aplicações em tempo real.

71 A arquitetura *cloud native* concentra-se em projetar e desenvolver aplicativos para serem executados em ambientes de nuvem e caracteriza-se pela adoção de práticas como microsserviços, contêineres e automação.

Julgue os itens subsequentes, a respeito da interoperabilidade entre aplicações.

72 OAuth, que é um protocolo utilizado para autorizar aplicações *web*, *desktop*, *mobile* e dispositivos em geral, não separa o papel do cliente do papel do proprietário do recurso.

73 Para alcançar a interoperabilidade entre aplicações, é necessário adotar padrões abertos e comuns, tais quais serviços *web*, REST, JSON, XML, OAuth e OpenID Connect.

Acerca de API Gateway, julgue o próximo item.

74 Considere-se que seja necessário criar uma API Gateway para um serviço *back-end* que responde a solicitações HTTP na rota `/api/fiscalização`. Nessa situação hipotética, na etapa de configuração da rota para o serviço de *back-end*, é necessário especificar, na API Gateway, o método HTTP (`GET`, `POST` etc.) que a aplicação *back-end* suporta, para que a integração funcione corretamente.

Tendo em vista que, no atual cenário de desenvolvimento de aplicações *web*, é essencial considerar princípios, como consistência e escalabilidade, e práticas, como automação do processo de implantação e integração do código-fonte, julgue os itens subsequentes.

75 As ferramentas DevOps incluem o Kubernetes, uma plataforma de orquestração de contêineres que permite gerenciar e escalonar aplicativos em contêineres em diferentes ambientes e pode ser integrada a outras ferramentas DevOps, como Docker e Jenkins.

76 Um exemplo prático de conteinerização de aplicação é a utilização do Docker para criar um ambiente consistente; nesse caso, é correto criar um arquivo Dockerfile por meio do comando `docker build`.

77 A adoção de tecnologias de conteinerização, como o Docker, permite à equipe orquestrar vários contêineres em conjunto, por exemplo, um contêiner para a aplicação Node.js e outro, para o banco de dados MongoDB.

78 O Ansible é uma ferramenta de integração contínua que permite automatizar o processo de construção, teste e implantação de *software* e pode ser integrado a outras ferramentas DevOps, como Git, Docker e Kubernetes.

Julgue os itens que se seguem, com relação a técnicas de integração e implantação contínua de código.

79 Embora as técnicas de integração contínua sejam essenciais para garantir a qualidade do *software* e a eficiência do processo de desenvolvimento, elas não possibilitam a realização de testes automatizados de aceitação.

80 Para configurar um projeto Freestyle no Jenkins, devem-se definir o *pipeline* de compilação, o teste e a implantação que serão executados quando o projeto for construído.

81 A implantação contínua é uma prática que envolve a entrega automatizada do *software* em diferentes ambientes, como ambientes de teste, pré-produção e produção, reduzindo o tempo necessário para implantar novas funcionalidades e corrigir *bugs*.

Acerca de HTTPRequest e JSON, julgue os próximos itens.

82 O processo de adição de comentários em arquivos JSON é igual ao das linguagens de programação, pois, assim como estes, tipicamente ignora comentários quando são executados.

83 O nó HTTPRequest é utilizado para interagir com um *web service* e está disponível nos seguintes modos de operação: desenvolvedor, *application integration suite*, padrão, avançado, *express*, escala e adaptador.

Acerca dos testes de *software* e das ferramentas para automatização de testes, bem como do desenvolvimento orientado por comportamento, julgue os itens que se seguem.

84 Em um teste de integração, cada uma das unidades é testada separadamente para se observar se elas funcionam de forma adequada.

85 No desenvolvimento orientado por comportamento (BDD), as palavras-chave utilizadas nos blocos que formam os cenários são *given*, *when* e *then*.

86 No particionamento de equivalências para a criação de casos de teste, devem ser consideradas apenas as partições válidas.

87 Em um teste funcional de *software*, os elementos de uma classe devem se comportar de maneira equivalente.

88 Na análise do valor limite, casos de teste podem ser derivados dos domínios de entrada e de saída.

Julgue os seguintes itens, a respeito dos métodos e das práticas ágeis para desenvolvimento de *software*.

89 No método Scrum, o *backlog* da *sprint* é um documento entregue pela equipe de desenvolvimento e que contém os resultados alcançados durante a *sprint*.

90 *To do*, *doing* e *done* são três estágios básicos do método Kanbam usados para caracterizar o andamento das atividades de um projeto de desenvolvimento de *software*.

91 Uma equipe *scrum* é formada pelos seguintes artefatos: um dono do produto, o *scrum master* e a equipe de desenvolvimento.

92 O *just in time* e o *poka yoke* são duas ferramentas utilizadas no método ágil XP para desenvolvimento de *software*.

93 A retrospectiva da *sprint* é a reunião que se faz para se avaliar o que foi feito na *sprint* anterior.

94 O dono do produto é responsável por garantir que os ritos do *scrum* sejam adotados corretamente.

No que concerne à elicitação e ao gerenciamento de requisitos, julgue os itens subsequentes.

95 Um requisito funcional pode ser expresso em termos de segurança e desempenho de um *software*.

96 A seguinte frase representa a sintaxe de uma história de usuário: como [*persona*], eu [quero], [para que].

97 As histórias de usuários podem ser divididas em partes menores chamadas de épicos.

98 Um critério de aceitação, depois de testado, significa que um requisito foi implementado corretamente.

99 Requisito é uma condição necessária que o sistema deverá realizar para atender a uma necessidade do usuário.

Julgue os itens a seguir, a respeito de inteligência artificial (IA) e *machine learning*.

100 *K-means* é um algoritmo de aprendizado não supervisionado, em que se calcula a distância entre os objetos da base e cada um dos centroides; em que se atribui cada objeto ao centroide mais próximo; e em que se classifica cada item para sua média mais próxima.

101 Nos algoritmos de aprendizado por reforço, o agente recebe uma recompensa atrasada na próxima etapa de tempo para avaliar sua ação anterior; seu objetivo, então, é maximizar a recompensa.

Com relação à *Low*/*No Code* e *robot process automation* (RPA), julgue os próximos itens.

102 A tecnologia RPA é caracterizada por plataformas de desenvolvimento que possuem interfaces gráficas e robóticas e tem o objetivo de possibilitar que o desenvolvedor construa seu projeto com a ajuda de robôs.

103 *Low*/*No Code* é uma técnica automática de programação que propicia a evolução de programas de computadores que resolvem (ou aproximadamente resolvem) problemas; ela manipula soluções corretas e incorretas, encoraja inconsistências e abordagens contraditórias e não apresenta uma variabilidade dinâmica lógica.

Acerca de criptografia simétrica e assimétrica, julgue o item subsequente.

104 A chave pública é gerada a fim de que o usuário possa divulgar o seu texto, antes de ele ser lido, para os leitores da mensagem.

Julgue o item subsecutivo, relativo a técnicas de refatoração de *software*.

105 O processo de refatoração de *software* melhora o *design* do código existente, mas não substitui nem altera o seu comportamento.

Julgue o item seguinte, relativo a protocolo HTTP/HTTPs.

106 HTPPs é a combinação do HTTP sobre a camada SSP (*secure sockets layer*), que é colocada entre a camada de aplicação e a camada de transporte, aceitando solicitações do navegador e enviando-as ao TCP (*trasmission control protocol*) para transmissão ao servidor.

Julgue os itens que se seguem, a respeito de Python, Java e orientação a objetos.

107 O código em Python a seguir apresentará um erro de execução.

```
r = 10
i = 7
c = complex(r, i) + (6 + 3j)
print(c)
```

108 Após ser executado, o código em Java a seguir apresentará 20 como resultado.

```
interface Foo{int x = 10;}
public class Acme {
  public static void main(String[] args) {
     Foo.x = 20;
     System.out.println(Foo.x);
  }
}
```

109 O resultado da execução do código em Python a seguir será
`Meu nome é Garfield.`

```
class Animal:

     nome = ""
     def patas(self):
          print("Tenho quatro patas")
class Gato(Animal):
     def display(self):
          print("Meu nome é", self.nome)

garfield = Gato()
garfield.nome = "Garfield"
garfield.patas()
garfield.display()
```

Com relação aos conceitos de Spring Boot, Net Core e *thread*, julgue os itens seguintes.

110 Spring Boot é uma tecnologia que se integra a ferramentas e linguagens de desenvolvimento *web* a fim de otimizar seu código, a partir de pequenos ajustes e trocas para deixar mais rápido o resultado do código.

111 *Thread* é um pequeno conjunto de instruções projetadas para serem agendadas e executadas pela CPU de forma independente do processo-pai.

112 NET Core é uma *framework* que trabalha com servidores *web* da Microsoft para integrar e automatizar os processos de *deploy* nos servidores, a fim de agilizar os processos e poupar os desenvolvedores desse trabalho.

Julgue os itens subsequentes, a respeito de HTML5, CSS3 e JavaScript.

113 O elemento HTML `<svg>` é um contêiner para gráficos SVG (*scalable vector graphics*), sendo SVG um formato de arquivo vetorial de imagem compatível com aplicações *web*.

114 O código JavaScript a seguir, ao ser executado, apresenta o valor `3` no console de desenvolvimento do navegador *web*.

```
<script>
  var cores = ["vermelho", "verde", "azul"];
  console.log(cores.length);
</script>
```

115 O código HTML/CSS a seguir apresenta Alo Mundo como resultado.

```
<html>
<p   style="text-transform:capitalize">   alo
mundo </p>
</html>
```

Acerca de primitivas de sincronização e *deadlocks*, julgue o próximo item.

116 No ambiente de programação, um semáforo é um sinal emitido pelo *hardware* para que o sistema operacional possa executar um laço com o objetivo de evitar um *deadlock*.

Com relação a conceitos de modelos de dados e de modelagem de dados, julgue os itens subsecutivos.

117 Em um sistema de banco de dados, a alteração do nível interno exige a revisão de nível conceitual equivalente.

118 Na construção de um banco de dados, o modelo conceitual utiliza os requisitos de negócio, que são decompostos em entidades, atributos e relacionamentos atômicos, e aplica regras para evitar redundâncias.

119 Na abordagem relacional, a propriedade de fechamento garante que a saída de qualquer operação possa ser usada como entrada de outra operação.

120 Como forma de garantir a integridade referencial, sempre que uma linha de uma tabela for removida, automaticamente as linhas de outras tabelas que fazem referência a ela também serão removidas.

121 Uma relação que esteja na quinta forma normal (5FN) está necessariamente livre de anomalias que sejam passíveis de serem eliminadas a partir de projeções.

Julgue os itens a seguir, a respeito de arquitetura de dados, metadados e linguagens de bancos de dados.

122 Para garantir o princípio da segurança de dados, que é um dos princípios da arquitetura de dados, o sistema deve ser rígido e ter suas regras de utilização e acesso fixadas e rigorosamente seguidas, sem espaço para alterações.

123 Os metadados possibilitam uma visão integrada do ambiente de dados, pois explicitam os inter-relacionamentos existentes entre os dados.

124 Comandos expressos em linguagem de definição de dados (DDL) são utilizados para criar estruturas de um banco de dados, e o seu processamento irá incluir ou alterar metadados desse mesmo banco de dados.

125 Usando-se SQL (*structured query language*), é possível unir o resultado de duas instruções `SELECT` quaisquer, por meio do operador `UNION`.

126 A manipulação de dados armazenados em um banco de dados é feita por meio de DML (*Data Manipulation Language*), linguagem na qual é possível apenas descrever os dados que se deseja acessar, sem especificar como obtê-los.

A respeito de propriedades e transações de bancos de dados, julgue os itens subsequentes.

127 Sistemas gerenciadores de banco de dados (SGBD) utilizam o recurso do bloqueio como forma de controlar a concorrência aos dados armazenados, porém esses bloqueios podem gerar *deadlocks*, que precisarão ser tratados.

128 Pode-se considerar que uma transação que atingiu o seu ponto de `COMMIT` garantiu suas propriedades de atomicidade e durabilidade.

129 No processamento de transações em um sistema de banco de dados, o uso de bloqueios (*locks*) tem o objetivo de garantir a propriedade da atomicidade das transações.

Julgue os próximos itens, relativos a NoSQL, *data lake*, ETL e governança de dados.

130 Segundo o DAMA-DMBok, a escolha de uma ferramenta para uso na governança de dados deve ser precedida pela definição de metas e requisitos gerais de governança.

131 Em um banco de dados NoSQL, o modelo de grafos é especialmente interessante quando a interconectividade dos dados é tão importante quanto os dados em si.

132 A arquitetura de um *data lake* pode variar de organização para organização, uma vez que muitas tecnologias podem ser usadas e combinadas de diferentes formas.

133 A fase de carga em um processo de ETL exige que os dados sejam carregados em lotes programados, como forma de garantir a eficiência do sistema como um todo.

Julgue os itens a seguir, relacionados a conceitos de nuvem computacional e suas tipologias.

134 O modelo IaaS (*Infrastructure as a Service*) fornece componentes de infraestrutura que são oferecidos em grupos de recursos; o usuário deve contratar um grupo de recursos pelo tempo que precisar.

135 O modelo PaaS (*Platform as a Service*), que é um conjunto de serviços de nuvem que organiza um ambiente para o desenvolvimento de aplicativos, permite que novas soluções sejam desenvolvidas, sendo necessário apenas configurar os itens de infraestrutura.

136 Um dos grandes benefícios do modelo SaaS é a possibilidade de o usuário acessar seu conteúdo de interesse pela Internet.

137 Para que uma plataforma computacional seja classificada como nuvem, é necessário que os recursos possam ser consumidos a partir de qualquer dispositivo conectado à Internet e que eles sejam agrupados para atender a vários consumidores por meio de um modelo *multi-tenant*.

138 Em um modelo de computação em nuvem do tipo IaaS, o provedor do serviço é responsável por proteger toda a infraestrutura, além do sistema operacional e dos dados dos clientes.

Julgue os itens seguintes, a respeito de modelo de implantação de nuvem computacional e algumas de suas propriedades.

139 Uma nuvem pública é um modelo de implantação em que os recursos são oferecidos por um provedor e compartilhados por organizações e indivíduos, tendo escalabilidade praticamente infinita.

140 A Internet das coisas apresenta como demanda a resposta rápida e a computação em nuvem atende adequadamente a esse tipo de demanda.

141 A principal diferença existente entre os modelos públicos e os modelos privados de implantação de nuvem é que os privados são de uso exclusivo de uma determinada organização e devem necessariamente ser operados e geridos pela própria organização.

142 Em um ambiente de computação em nuvem, a elasticidade oferece às organizações a possibilidade de diminuir ou aumentar as capacidades computacionais de forma dinâmica.

143 Os aplicativos executados em uma nuvem computacional têm garantida a disponibilidade contínua, independentemente das cargas flutuantes.

144 A escalabilidade em nuvem computacional, em sua dimensão horizontal, significa tornar um componente maior ou mais rápido para lidar com uma carga maior.

Acerca de IaC (*Infrastructure as Code*), julgue os itens subsecutivos.

145 A escolha de uma solução de IaC com abordagem declarativa traz como principal desvantagem a necessidade de se ter um administrador qualificado para configurar e gerenciar a solução.

146 O principal objetivo de se escolher automatizar a infraestrutura por meio de IaC é implantar uma infraestrutura mutável, dinâmica.

No que se refere a componentes da arquitetura em nuvem, julgue os itens subsecutivos.

147 Em um ambiente de nuvem computacional, uma forma de garantir a disponibilidade dos dados é a sua replicação em regiões distintas, que é feita automaticamente pelo serviço de nuvem.

148 Uma zona de disponibilidade é composta por um conjunto de *datacenters* que não são compartilhados com outras zonas de disponibilidade.

149 Para definir a região que será utilizada pela nuvem computacional, os fatores que precisam ser observados incluem as regras de conformidade.

150 O critério decisivo utilizado para definir a distribuição geográfica de *datacenters* de uma mesma zona de disponibilidade é a possibilidade de ocorrência de desastres naturais.

Espaço livre